CSCE 20003 -Data Structures Exam Three Study Guide

This study guide covers the core linear data structures—Linked Lists, Stacks, and Queues—examining their structural anatomy, algorithmic trade-offs, underlying memory models, and code-level mechanics.

---

1. Linked Lists (Singly, Doubly, Circular)

Linked lists store elements in non-contiguous memory nodes linked via explicit references (pointers/references).

Structural Variants

- **Singly Linked List:** Each node contains a data payload and a single `next` pointer. Traversal is strictly unidirectional (head to tail). Searching, inserting, or deleting at arbitrary positions requires $O(n)$ traversal.
- **Doubly Linked List:** Each node contains data, a `next` pointer, and a `prev` pointer. Enables bidirectional traversal and $O(1)$ deletion given a direct reference to the target node (since you can access its predecessor and successor immediately).
- **Circular Linked List:** The tail node's pointer references the head node (singly) or both head and tail are mutually linked (doubly). Eliminates `null` termination checks, useful for round-robin schedulers or continuous buffers.

Core Time Complexities

Operation	Singly Linked (Head/Tail)	Doubly Linked	Circular
Prepend (Insert Head)	$O(1)$	$O(1)$	$O(1)$
Append (Insert Tail)	$O(1)$ (with tail ptr) / $O(n)$	$O(1)$ (with tail ptr)	$O(1)$ (with tail ptr)
Delete Head	$O(1)$	$O(1)$	$O(1)$
Delete Tail	$O(1)$ (with tail ptr) / $O(n)$	$O(1)$ (with tail ptr)	$O(n)$
Search by Value	$O(n)$	$O(n)$	$O(n)$

---

2. Stacks (List- vs. Array-Based)

A Stack operates on a **Last-In, First-Out (LIFO)** policy. All insertions (`push`) and deletions (`pop`) occur strictly at one end, designated as the *top*.

Array-Based Implementation

- **Mechanism:** Backed by a contiguous block of memory (dynamic array). A `top` index tracks the index of the current top element.
- **Pros:** Excellent cache locality; direct index access.
- **Cons:** Fixed initial capacity requires resizing (allocating a larger array and copying elements, resulting in $O(n)$ worst-case time for push, though amortized time is $O(1)$).

Linked-List-Based Implementation

- **Mechanism:** Backed by a singly linked list where the `head` pointer acts as the stack's `top`. Pushing creates a new node at the head ($O(1)$); popping removes the head node ($O(1)$).
- **Pros:** Dynamic sizing with no reallocation overhead or wasted capacity; strictly bounded memory usage per node.
- **Cons:** Higher memory overhead per element due to pointer storage; poor cache locality due to scattered heap allocation.

---

3. Queues (List- vs. Array-Based)

A Queue operates on a **First-In, First-Out (FIFO)** policy. Elements enter at the *rear* (`enqueue`) and leave from the *front* (`dequeue`).

Array-Based Implementation (Circular Buffer)

- **Mechanism:** To prevent shifting all elements on every dequeue (which takes $O(n)$ time), use a **circular array** with `front` and `rear` indices managed using modulo arithmetic: `index = (index + 1) % capacity`.
- **Handling Capacity:** When `(rear + 1) % capacity == front`, the queue is full, triggering a resize operation.

Linked-List-Based Implementation

- **Mechanism:** Maintained using a singly linked list with explicit `head` (front) and `tail` (rear) pointers.
- **Enqueue:** Append at the `tail` and update `tail` pointer in $O(1)$ time.
- **Dequeue:** Remove from the `head` and update `head` pointer in $O(1)$ time.

---

4. Architectural Trade-Offs: Contiguous vs. Linked Allocation

Metric	Array / Contiguous Allocation	Linked Node Allocation
Memory Overhead	Low (only data storage; occasional unused capacity in dynamic arrays).	High (extra memory consumed by `next`/`prev` pointers per node).
Cache Locality	**High.** Elements reside in contiguous memory, maximizing CPU cache line hits.	**Low.** Nodes are scattered across heap memory, leading to cache misses.
Resizing Cost	Amortized $O(1)$, but expensive worst-case reallocation and copying steps.	True $O(1)$ dynamic allocation per node; no large block reallocations.
Access Time	$O(1)$ random access via indexing.	$O(n)$ sequential traversal required to reach arbitrary indices.

---

5. Code Tracing & Completion Pitfalls

Expect exam problems requiring you to trace pointer manipulations or complete missing algorithmic blocks. Watch out for these critical failure modes:

1. **Lost References (Memory Leaks / Broken Chains):** When inserting a node into a linked list, *always* attach the new node's pointer to its successor *before* breaking the existing link. Reversing this order severs access to the remainder of the list.
2. **Null Pointer Exceptions (NPE):** Always check boundary conditions before dereferencing:
    - Is the list empty (`head == null`)?
    - Are you operating on a single-element list (`head == tail`)?
    - Did your traversal pointer hit `null` prematurely?
3. **Off-by-One Errors in Circular Buffers:** When implementing array-based queues or stacks, ensure empty vs. full states are distinguishable (e.g., keeping a separate `size` counter or leaving one slot empty in circular arrays).